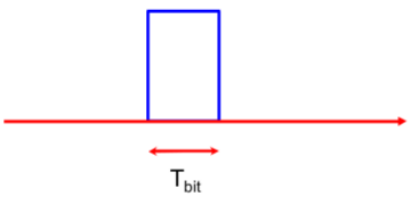
6_19_marzo

Throughput

il throughput è un indice della capacità di trasmissione in un canale di comunicazione, più è alto più dati si possono far passare in quel canale:

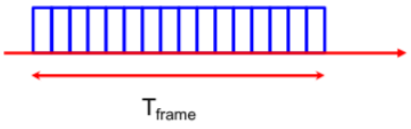
$$Throughput = \frac{Numero\ \ di\ \ bit\ \ (o\ \ byte)\ \ trasmessi\ \ con\ \ successo}{tempo\ \ impiegato}$$



BW = 10 Mbps = 10^7 bps

in 1 s => 10.000.000 bits

1 bit => T_{bit} = 0.1 µs



BW = 10 Mbps

Frame of 1500 bytes = 12000 bits

Time to generate a frame: T_{frame} = 1200 µs

Consideriamo un canale con *Bandwidth* pari 10 Mbps ovvero:

$$BW = 10 \text{ Mbps } = 10^7 \text{ bit per secondo}$$

da questo ne traiamo che:

- in $1s \rightarrow 10000000\ bits$
- in $0.1\mu s \rightarrow 1\ bits$
  Se volessimo inviare un Frame di 1500 bytes ovvero 12000 bits impieghiamo:

  $$T_{\text{frame}} = \frac{\text{Dimensione del frame (in bit)}}{\text{Bandwidth (in bit/s)}}$$

  Ad esempio per noi il tempo di frame sarebbe di 1.2 millisecondi

il nostro segnale si deve propagare da A a B, quindi dobbiamo mettere in considerazione anche il tempo di propagazione del segnale, di seguito in dettaglio:

$$T_{\text{propagazione}} = \frac{\text{Spazio di trasmissione}}{\text{Velocità di trasmissione}}$$

Con questa informazione possiamo calcolare anche il numero di bit che possono stare contemporaneamente nel canale:

$$\text{Bandwidth (in bit/s)} \cdot T_{\text{propagazione}}(\text{in s})$$

T_{frame}	$T_{propagazione}$
Il tempo impiegato dal PC per immettere tutto il frame nel canale	Il tempo impiegato dai dati per percorrere tutto il canale fino a destinazione

Date tutte queste informazioni possiamo calcolare il **throughput teorico** che è:

$$T = T_{\text{frame}} + T_{\text{propagazione}}$$

Quindi il **throughput teorico** sarebbe il tempo impiegato per immettere tutti i dati + quello per percorrere il canale fino a destinazione

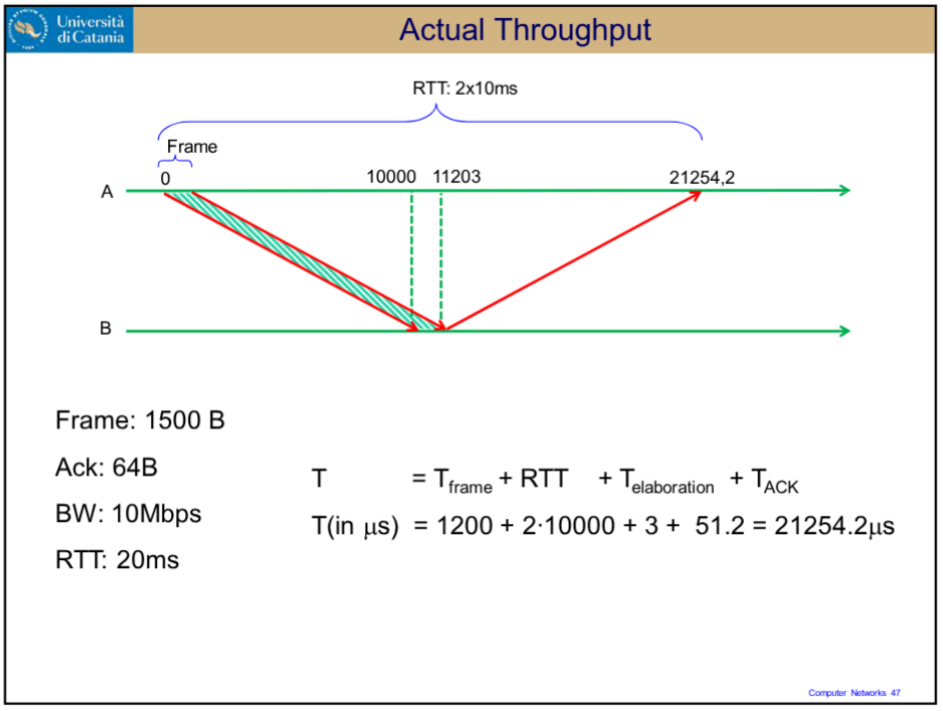
Per calcolare il **throughput reale** dobbiamo considerare il tempo di ACK e il tempo di RTT(Round-Trip Time) ovvero il tempo di andata e ritorno (quindi il $T_{propagazione}$ dell'andata + quello del ritorno dell' ACK), la formula diventa:

$$T = T_{\text{frame}} + RTT + T_{\text{elaborazione}} + T_{\text{ACK}}$$

$$T_{ACK} = \frac{\text{Dimensione dell'ACK (in bit)}}{\text{Bandwidth (in bit/s)}}$$

il tempo di elaborazione è un tempo teorico (molto piccolo) che introduciamo perché il computer B dopo aver ricevuto un messaggio lo legge e crea un messaggio di ACK e lo fa nel tempo di elaborazione che è piccolo ma non 0

RTT lo dà l'esercizio



In questa maniera la reale velocità del canale non è molto alta, si decide quindi di implementare un meccanismo di pipeline...

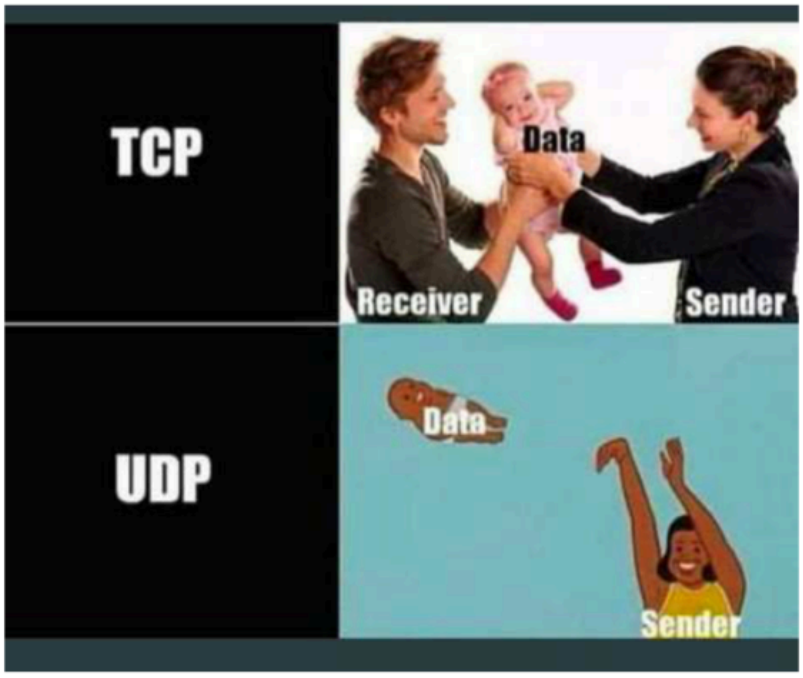
La modalità pipeline ci consente di aumentare il throughput di una rete. Invece di inviare un singolo frame ne inviamo n e aspettiamo n ACK, capiamo subito che è molto importante gestire i casi in cui il frame si rovina e l'ACK di risposta non arriva, usiamo principalmente questi due algoritmi:

- **Go-Back-N (GBN)**: *Se si perde un pacchetto, si rimanda tutto da quel punto in poi.*
  - **Finestra di invio:** Il mittente può inviare fino a N pacchetti senza aspettare conferma (ACK).
  - **Ricezione:** Il destinatario accetta i pacchetti solo in ordine rigoroso. Se arriva un pacchetto fuori sequenza, lo scarta.
  - **Gestione degli errori:** Se il pacchetto 3 va perso nel canale, il destinatario ignorerà i pacchetti 4, 5, ecc. Quando il timer del mittente scade per il pacchetto 3, il mittente ritrasmette il pacchetto 3 e tutti i pacchetti successivi già inviati (torna indietro a N).
- **Selective repeat**: *Se si perde un pacchetto, si rimanda SOLO quello perso.*
  - **Finestra di invio:** Simile al GBN, il mittente invia fino a N pacchetti senza aspettare l'ACK.
  - **Ricezione:** Il destinatario accetta i pacchetti fuori ordine e li **salva in un buffer**, inviando un ACK individuale per ogni pacchetto ricevuto correttamente.
  - **Gestione degli errori:** Se il pacchetto 3 va perso ma il 4 e il 5 arrivano, il destinatario memorizza 4 e 5. Quando scade il timer del mittente per il pacchetto 3, il mittente ritrasmette solo e unicamente il pacchetto 3.

Usando la pipeline si riesce ad aumentare la quantità di dati trasmessi.

Rumore: dobbiamo mettere in considerazione anche il rumore, ovvero il disturbo al nostro segnale durante la sua trasmissione.

Meme tra TCP e UDP



TCP

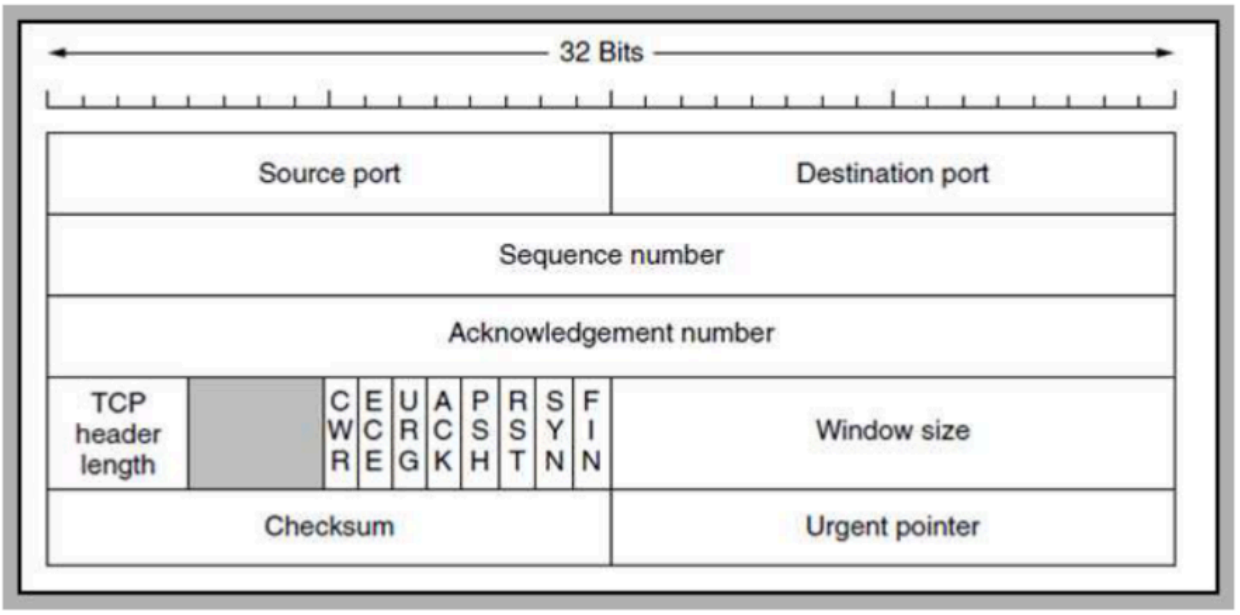
Nella ricostruzione delle versioni di TCP si dice ci siano 4 versioni, ma non è vero è una sola quella del 1981. Il TCP ha diverse funzionalità (feature):

- Indirizzamento/Multiplexing
- Creazione, Gestione e Terminazione della Connessione
- Gestione e Impacchettamento dei Dati
- Trasferimento Dati
- Fornitura di Servizi di Affidabilità e Qualità della Trasmissione
- Fornitura di Funzionalità di Controllo del Flusso e Prevenzione della Congestione

L'ultima è quella sulla quale stanno lavorando i ricercatori, per migliorarla. Ci sono delle cose a cui TCP non provvede:

- Non offre sicurezza
- Non garantisce una comunicazione
- Non mantiene i confini del messaggio (*message boundaries*), queste servono per riuscire ad interpretare e ricostruire messaggi.

Di seguito la struttura del pacchetto TCP:



Troviamo diverse informazioni:

- **Destinatione e Source Port**
- **Sequence Number:** Campo di 32 bit che indica il numero di sequenza del primo byte dei dati contenuti nel segmento TCP.
  Esempio pratico: Supponiamo che tu debba inviare 500 byte di dati. Il pacchetto parte con un Sequence Number impostato a `1000`. Questo significa che il pacchetto contiene i byte che vanno dal numero `1000` al `1499`.
- **Acknowledgment Number (ACK):** Campo di 32 bit che indica il numero di sequenza del prossimo byte che il ricevitore si aspetta di ricevere (conferma cumulativa).
  Esempio pratico: Il destinatario riceve il tuo pacchetto con i byte dal `1000` al `1499`. Per dirti che è andato tutto bene, ti manda indietro un messaggio con l'ACK impostato a `1500`. Ti sta letteralmente dicendo: "Perfetto, ho tutto fino al 1499. Ora sto aspettando che mi mandi il byte 1500".
- **TCP header length (chiamato anche Data Offset):** Questo campo indica quanto è lungo l'intero header TCP. È fondamentale perché serve a dire al computer ricevente: "L'header finisce qui, da questo punto in poi iniziano i dati veri e propri (il payload)".
- **Spazio grigio (Reserved):** Sono bit "messi da parte" per sviluppi futuri del protocollo. Attualmente vengono impostati a zero e ignorati
- **Flag** Quella fila di lettere in verticale rappresenta i Flag, ovvero dei piccoli interruttori di 1 bit che indicano lo stato del pacchetto o danno istruzioni speciali.
- **Window size (Dimensione della finestra):** È il cuore del *controllo di flusso* che avevamo menzionato prima. Indica quanti byte il computer ricevente è in grado di accettare in quel momento. Se il ricevente è sovraccarico, abbasserà questo valore per dire al mittente: "Rallenta, non ho più spazio in memoria!"
- **Checksum:** È un valore matematico di controllo per la correzione degli errori

- - **Urgent pointer (Puntatore urgente):** Viene letto solo se il flag **URG** è attivo. Indica esattamente a quale byte finiscono i dati urgenti all'interno del pacchetto.

Tutto questo rende TCP affidabile ma non molto veloce, tuttavia anche il througput è un fattore da tenere in considerazione, per analizzarlo dobbiamo analizzare una serie di parametri:

- **MSS(Maximum Segment Size)**: Indica la dimensione massima dei dati che possono essere inviati in un singolo segmento TCP
- **MTU(Maximum Tramission Unit)**: → $MSS + HeaderTCP + HeaderIP$

Un'altro dato importante è RTT(Round Trip Time), questo tempo viene continuamente stimato in questo modo:

- Per ogni comunicazione viene misurato il tempo che trascorre tra l'invio del pacchetto e il ritorno di un ipotetico ACK
- Viene fatta la media di tutti i tempi salvati
  è importante che RTT siano stimato bene perché:
- RTT troppo lungo: si ha una reazione troppo lenta alla perdita di pacchetti
- RTT troppo corto: gli ACK potrebbero non arrivare in tempo
  il calcolo del RTT viene fatto ad ogni invio di pacchetto, in modo che TCP sappia quanto aspettare. Il tipo di media che viene usata si chiama: **Exponential Weighted Moving Average**. (Moving perché si muove un valore avanti ad ogni pacchetto)

Deviazione standard RTT
Abbiamo detto che il tempo RTT viene calcolato attraverso una media dei vari tempi che si sono registrati, ma non sempre i tempi registrati sono tutti simili tra loro (es. 1 ms, 1.12 ms, 1.09 ms) potrebbero infatti esserci oscillazioni molto grandi tra i vari tempi misurati, (es. 1 ms, 0.3 ms, 3 ms) quindi il calcolo della deviazione standard RTT è proprio il calcolo di queste oscillazioni molto grandi:

$$DevRTT = (1 - \beta) \cdot DevRTT + \beta \cdot |SampleRTT - EstimatedRTT|$$

Un valore tipico di β è $\frac{1}{4} = 0.25$
Dove:

- β è un peso fissato (0.25 di solito)
- Sample RTT è il tempo misurato con l'ultimo pacchetto inviato
- Estimated RTT è il tempo storico (ovvero la media fino a quel momento)

A differenza dei modelli teorici RDT (che prevedono un timer per ogni singolo pacchetto), TCP riduce la complessità del sistema utilizzando **un solo timer di ritrasmissione globale**.

Funziona in questo modo: quando TCP riceve un ACK che conferma un pacchetto, resetta il timer. Se ci sono altri dati già inviati ma non ancora confermati, il timer viene immediatamente riavviato per monitorare il segmento "più vecchio" ancora in volo (ad esempio, il segmento successivo *n+1* in attesa di ACK).

Per adattarsi alle condizioni della rete, TCP calcola dinamicamente il timer di ritrasmissione attraverso questi 4 passaggi:

1. **SampleRTT (Campione):** Misurazione del tempo effettivo trascorso tra l'invio di un singolo segmento e la ricezione del suo ACK.
2. **EstimatedRTT (Tempo stimato):** Media mobile ponderata dei campioni precedenti. Serve a "smussare" i picchi e le anomalie temporanee della rete.
3. **DevRTT (Deviazione):** Calcolo della variabilità del RTT. Funge da margine di sicurezza per evitare timeout prematuri.
4. **RTO (Retransmission Timeout):** Il timer effettivo da utilizzare, calcolato come la somma del tempo stimato più il margine di sicurezza

   $$EstimatedRTT + 4 * DevRTT$$

   la formula dell'RTO è progettata per essere prudente, il timer finale calcolato risulta spesso **molto lungo**. Se un pacchetto viene perso, aspettare la scadenza di questo timer causa un lungo periodo di inattività e rallenta la trasmissione. Per mettere una pezza a questo problema di latenza, TCP utilizza un meccanismo chiamato **Fast Retransmit** (Ritrasmissione Rapida):

- Se il ricevitore rileva un "buco" nei pacchetti, continua a mandare ACK con il numero del pacchetto mancante.
- Se il mittente riceve **3 ACK duplicati** di fila, deduce con certezza che il pacchetto successivo è andato perso.
- A questo punto il mittente **ritrasmette immediatamente il pacchetto**, senza aspettare che il lungo timer RTO scada.